

{ 04 UCS }

Lab 3: Uniform Cost Search

Now the edges have costs. find the cheapest path from A to D.

$$A \rightarrow B \Rightarrow 1 ; B \rightarrow D \Rightarrow 5 ; C \rightarrow D \Rightarrow 1$$

$$A \rightarrow C \Rightarrow 4 ; B \rightarrow C \Rightarrow 2$$

There are exactly 3 routes:

$$A \rightarrow B \rightarrow D \text{ costs } 6$$

$$A \rightarrow C \rightarrow D \text{ costs } 5$$

$$A \rightarrow B \rightarrow C \rightarrow D \text{ costs } 4$$

Answer this: The cheapest root is the one with the most hops. BFS would return a 2-hop route costing 6. why must the goal be tested when a path is popped, not when it is generated.

⇒ Aim: — To find the cheapest path from A to D using UCS

• Given Graph: — $A \rightarrow B \Rightarrow 1 ; B \rightarrow D \Rightarrow 5$
 $A \rightarrow C \Rightarrow 4 ; B \rightarrow C \Rightarrow 2$

$$C \rightarrow D \Rightarrow 1$$

There are 3 possible routes from A to D:

$$A \rightarrow B \rightarrow D \quad \text{cost} = 1 + 5 = 6$$

$$A \rightarrow C \rightarrow D \quad \text{cost} = 4 + 1 = 5$$

$$A \rightarrow B \rightarrow C \rightarrow D \quad \text{cost} = 1 + 2 + 1 = 4$$

Therefore cheapest path $\Rightarrow A \rightarrow B \rightarrow C \rightarrow D$

Minimum cost $= 4$

- What is UCS? — UCS is an algorithm used to find cheapest path from a starting point to a goal.

It does not care about the no of steps.

It only cares about total path cost

- UCS vs BFS

BFS :- BFS mainly looks at the no of hops/edges. (Uses Queue)

Eg $A \rightarrow B \rightarrow D$ has 2 hops

BFS may choose it because it has only 2 hops.

But its cost is : $1 + 5 = 6$

UCS :- UCS looks at the total cost. (Uses Priority Queue)

Eg It finds : $A \rightarrow B \rightarrow C \rightarrow D$

cost : $1 + 2 + 1 = 4$

Even though it has 3 hops, it is cheaper.

- How UCS works :-
- i) UCS uses a Priority Queue.
 - ii) The priority is the path cost.
 - iii) The path with the smallest cost gets the highest priority.

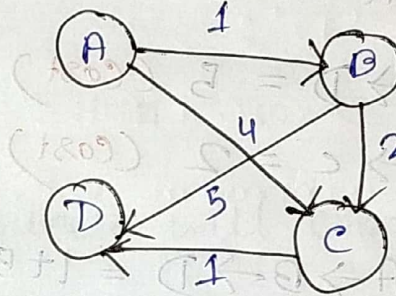
eg

Path	Cost
A → B	1
A → C	4

UCS choose: A → B
Because: $1 < 4$

Step-by-Step UCS :-

Our Graph :-



we want to go: A → D

i) Step 1: Start at A :-

Initially we have: Priority Queue: (A)
Cost = 0

We start with A; Path = A

Cost = 0

ii) Step 2: Pop A :- we remove A from the Priority queue.

Current Path = A
Cost = 0

A has 2 neighbours: A → B = 1 (cost)

A → C = 4 (cost)

So, we add both paths.

Priority Queue becomes : $A \rightarrow B$ 1
 $A \rightarrow C$ 4

The cheapest path is : $A \rightarrow B$

iii) Step 3 : Pop $A \rightarrow B$:-

The cheapest path is : $A \rightarrow B$
Cost = 1

So, UCS removes it from the queue.

Now, we look at B.

B has : $B \rightarrow D = 5$ (cost)
 $B \rightarrow C = 2$ (cost)

Therefore : $A \rightarrow B \rightarrow D = 1 + 5 = 6$ (cost)

$A \rightarrow B \rightarrow C = 1 + 2 = 3$ (cost)

Add them to Priority Queue.

Now, the queue contains : $A \rightarrow B \rightarrow C$ 3

$A \rightarrow C$ 4

$A \rightarrow B \rightarrow D$ 6

The cheapest one is : $A \rightarrow B \rightarrow C$

iv) Step 4 : Pop $A \rightarrow B \rightarrow C$:-

The Cheapest Path is : $A \rightarrow B \rightarrow C$

So, UCS removes it from the queue.

Now, look at C.

c has: $C \rightarrow D = 1$ (cost)

so, $A \rightarrow B \rightarrow C \rightarrow D = 1 + 2 + 1 = 4$ (cost)

Add it to the priority queue.

now, the queue contains: $A \rightarrow B \rightarrow C \rightarrow D$ 4

$A \rightarrow C$

4

$A \rightarrow B \rightarrow D$ 6

v) Step 5: Pop the cheapest path :-

The cheapest paths have cost:

$A \rightarrow B \rightarrow C \rightarrow D$ 4

$A \rightarrow C$

our priority queue will choose 1 according to its tie-breaking order.

Suppose: $A \rightarrow B \rightarrow C \rightarrow D$, is popped first.

now the last node is D.

D is our goal.

Therefore we stop.

vi) Final Answer :- cheapest path: $A \rightarrow B \rightarrow C \rightarrow D$

Total cost: $1 + 2 + 1 = 4$

Python Code :-

1. `import heapq`

2. `graph = {`

3. `'A': [( 'B', 1 ), ( 'C', 4 )],`

4. 'B' : [('D', 5), ('C', 2)],
5. 'C' : [('D', 1)]
6. 'D' : []
7. 3
8. pq = [(0, 'A', ['A'])]
9. while pq :
10. cost, node, path = heapq.heappop(pq)
11. if node == 'D' :
12. print("cheapest path:", path)
13. print("minimum cost:", cost)
14. break
15. for next_node, edge_cost in graph[node]
16. new_cost = cost + edge_cost
17. new_path = path + [next_node]
18. heapq.heappush(pq, (new_cost, next_node, new_path))

Output:- Cheapest Path : ['A', 'B', 'C', 'D']

Minimum Cost : 4

● Code Explanation :-

1. Imports heapq, which helps us make a Priority Queue.
2. Starts sorting our graph

3. From A :
 - $A \rightarrow B$ costs 1
 - $A \rightarrow C$ " 4
4. From B :
 - $B \rightarrow D$ " 3
 - $B \rightarrow C$ " 2
5. From C :
 - $C \rightarrow D$ " 1
6. D has no next node.
7. Ends the graph

8. creates the Priority Queue.

It stores : 0 $\rightarrow$ total cost

'A' $\rightarrow$ current node

['A'] $\rightarrow$ Path taken

So, we start at : A

Cost = 0

9. Keep running while the Priority Queue is not empty.

10. Takes out the cheapest Path from the Priority Queue.

Eg Cost = 1

Node = B

Path = $A \rightarrow B$

heappop() = remove the cheapest item.

11. Checks : "Have we reached D?"

12. Prints the cheapest Path



Eg Cheapest Path: ['A', 'B', 'C', 'D']

13. Prints the total cost.

Eg Minimum cost: 4

14. Stop the program.

why? Because D has been popped as the Cheapest Path, so we know it is the cheapest route.

15. Looks at all neighbours of the current node.

Eg if: node = B
then it checks: $B \rightarrow D = 5$
 $B \rightarrow C = 2$

16. Calculate the new total cost

Eg $A \rightarrow B = 1$
 $B \rightarrow C = 2$

new cost = $1 + 2 = 3$

17. Adds the new node to the path.

Eg old path $\Rightarrow A \rightarrow B$

next node $\Rightarrow C$

new path $\Rightarrow A \rightarrow B \rightarrow C$

18. Push the new path into the Priority Queue.
It stores: new cost
" node
" path

Then UCS will later choose the path with the smallest cost.

● Why do we check the goal when it is popped?

Because generated does not mean cheapest.

i) when a goal is generated, its path may still be expensive.

ii) UCS puts it in the priority queue and waits.

iii) when the goal is popped, it has the lowest total cost.

Eg $A \rightarrow B \rightarrow D = 6 \rightarrow$ generated 1st ~~X~~

$A \rightarrow B \rightarrow C \rightarrow D = 4 \rightarrow$ cheaper ✓

So, the goal when popped, not generated, to guarantee the cheapest path.